

PROGRAMMING SERIES

Core Java

Every essential concept, explained in plain English — with deep theory, runnable examples and hand-drawn diagrams.

Beginner → Interview Ready | Java 21 | 18 Chapters

Table of Contents

- [01 Java, the JVM, JDK & JRE — How Code Actually Runs](#)

- [02 Variables, Data Types & Operators](#)

- [03 Control Flow — Decisions & Loops](#)

- [04 The Four Pillars of OOP](#)

- [05 Classes, Objects & Constructors](#)

- [06 static, final, this & super](#)

- [07 Inheritance, Abstract Classes & Interfaces](#)

- [08 Packages & Access Modifiers](#)

- [09 String & Immutability](#)

- [10 Arrays](#)

- [11 Exception Handling](#)

- [12 Collections Framework](#)

- [13 Generics](#)

- [14 Multithreading & Thread Life Cycle](#)

- [15 File I/O](#)

- [16 Inner Classes](#)

- [17 Lambdas & the Stream API](#)

- [18 Memory Management — Stack, Heap & Garbage Collection](#)

Java, the JVM, JDK & JRE — How Code Actually Runs

Java's one big promise is "**write once, run anywhere**". That sentence hides an elegant machine. Once you understand it, everything else in Java becomes easier.

The problem Java solved

In older languages like C, you compile your source into machine code for one specific operating system. A program built for Windows will not run on Mac. You had to recompile for every platform. Java broke that link.

The three-step journey of a Java program

Java source is never turned directly into machine code. It is first turned into a **middle language** called bytecode, which is platform independent. Then a small program called the **JVM** translates that bytecode into machine code for whatever machine it is currently running on.

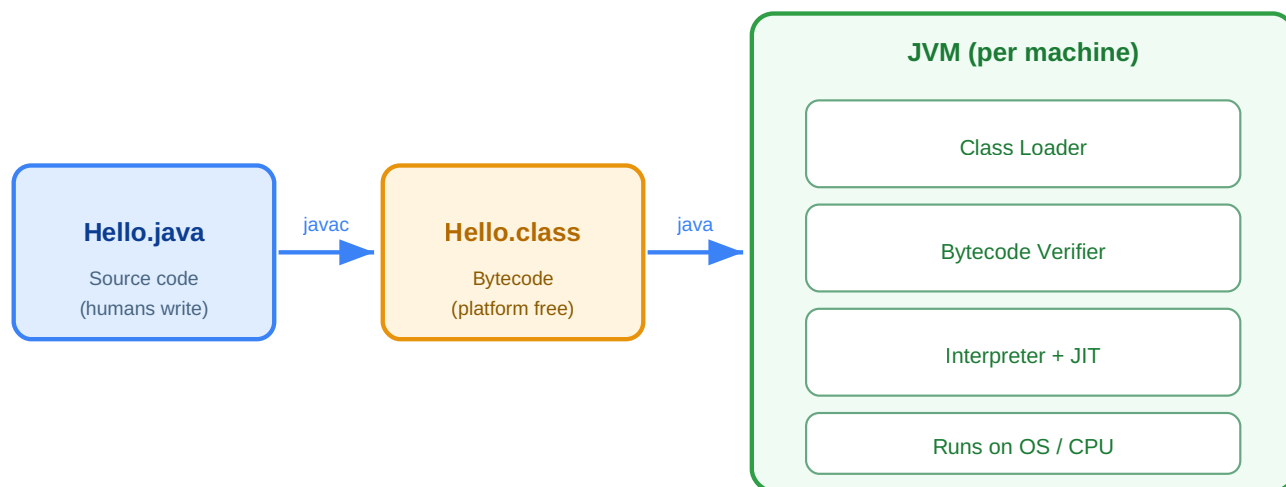


Figure 1.1 — Source is compiled once into bytecode; each JVM executes that same bytecode on its own machine.

Why bytecode is the secret ingredient

- **javac** (the compiler) does not care about your operating system — it only produces `.class` bytecode.
- Any device with a JVM can run that file: Windows, Mac, Linux, even a smart card.
- The JVM is the only platform-specific part, and someone else already wrote it for you.

JDK vs JRE vs JVM — the famous confusion

These three names are nested boxes. The **JVM** is the engine. The **JRE** is the engine plus the standard libraries it needs to run programs. The **JDK** is the JRE plus the developer tools (`javac` , `jar` , `javadoc` , debugger).

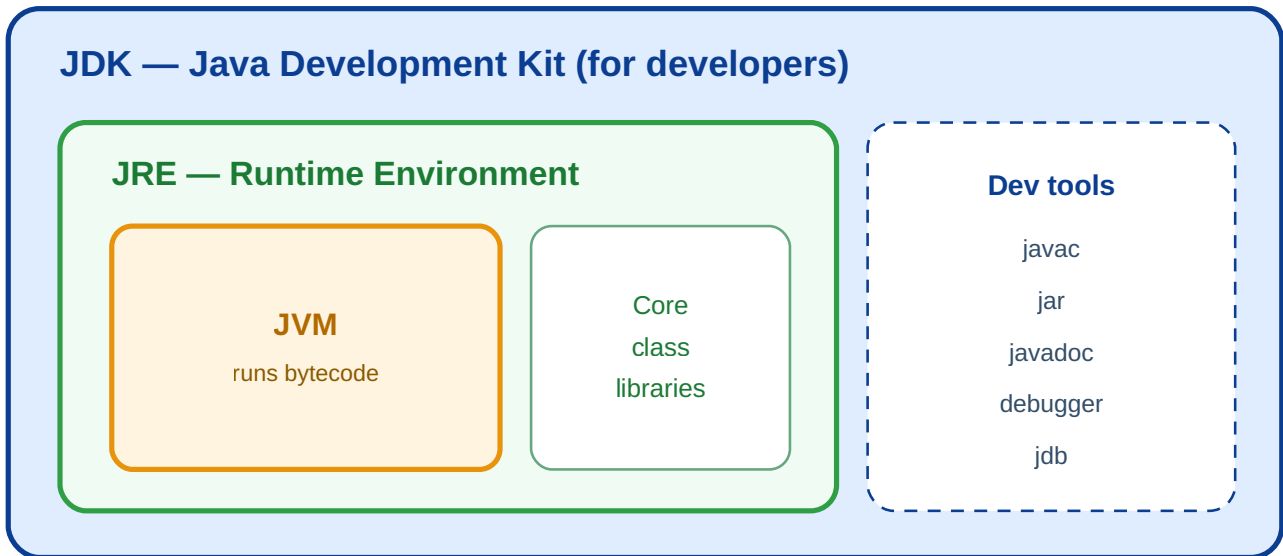


Figure 1.2 — $\text{JDK} \supset \text{JRE} \supset \text{JVM}$. If you only want to run apps, JRE is enough; to build apps, you need the JDK.

Simple analogy

JVM = a car engine. JRE = a complete car you can drive. JDK = a car plus a full mechanic's garage so you can build and repair cars.

Interpreted and compiled — at the same time

Java is neither purely interpreted nor purely compiled. The JVM starts by **interpreting** bytecode line by line (fast to start, slower to run). It also watches which methods are used most; those "hot" methods get **JIT-compiled** into native machine code on the fly, so they run at full speed. This hybrid design gives Java both quick startup and long-run performance.

Common myth

"The JVM interprets everything, so Java is slow." Not true today — just-in-time compilation makes long-running Java code nearly as fast as C++ for many workloads.

Your first program, dissected

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

```
Output:  
Hello, Java!
```

- `public` — visible to the JVM launcher.
- `class Hello` — everything in Java lives inside a class; the file must be named `Hello.java`.
- `public static void main(String[] args)` — the exact entry point the JVM looks for. `static` means the JVM can call it without creating an object.
- `System.out.println(...)` — print a line to the console.

How the JVM is structured internally

Inside the JVM there are three big areas: the **class loader** (brings `.class` files into memory), the **runtime data areas** (where memory lives while the program runs), and the **execution engine** (which actually runs the code).

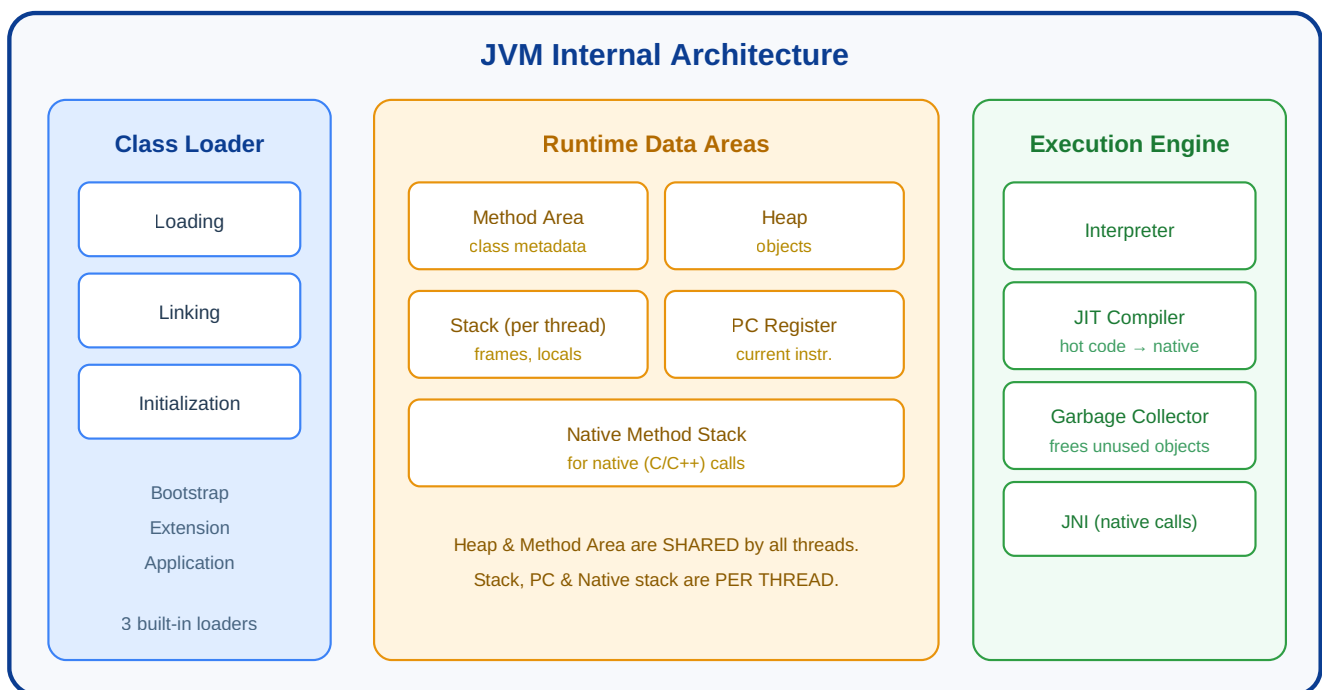


Figure 1.3 — The three pillars of the JVM: class loading, memory areas, and execution.

Key vocabulary you should own

Term	Meaning in one line
Source code	The <code>.java</code> file you type.
Bytecode	Portable middle-language in the <code>.class</code> file.
JVM	The program that loads and runs bytecode.
JRE	JVM + core libraries, enough to run.
JDK	JRE + tools, enough to build.
JIT	Just-In-Time compiler that turns hot bytecode into native code.

Classpath	The list of places the JVM searches for classes.
-----------	--

Variables, Data Types & Operators

A variable is a named box in memory. Its **data type** decides how big the box is and what kind of value fits inside.

Two families of types: primitive and reference

Primitive types store the actual value directly. There are exactly eight of them, and each has a fixed size — which is why Java behaves the same on every machine.

Type	Size	Stores	Example
byte	1 byte	whole number −128...127	byte b = 100;
short	2 bytes	whole number ±32,767	short s = 30000;
int	4 bytes	whole number ±2.1 billion	int n = 42;
long	8 bytes	very large whole number	long L = 90000000000L;
float	4 bytes	decimal (7 digits precision)	float f = 3.14f;
double	8 bytes	decimal (15 digits precision)	double d = 3.14159;
char	2 bytes	one Unicode character	char c = 'A';
boolean	1 bit	true or false	boolean ok = true;

Reference types (objects, arrays, Strings) do not hold the value itself. They hold the address of where the object lives in the heap. Think of a primitive as a house, and a reference as a slip of paper with the house's address written on it.

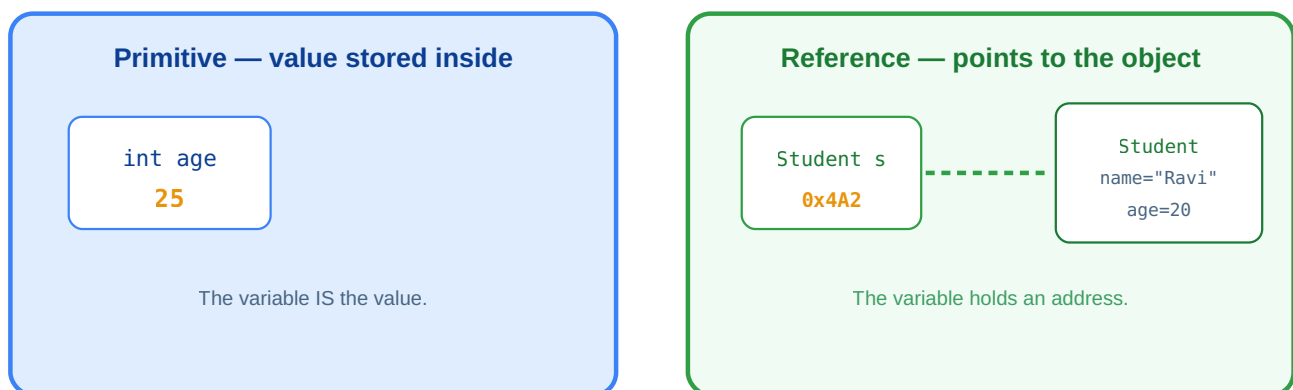


Figure 2.1 — Primitives carry the value; references carry a pointer to a heap object.

Declaring and assigning

```
int count;           // declare (box created, no value yet)
count = 10;          // assign
```

```
double price = 99.5; // declare + assign in one line
final int MAX = 100; // constant: cannot be reassigned
```

Default values

Instance and static fields get automatic defaults (`0` , `0.0` , `false` , `null`). **Local variables inside methods do not** — using one before assigning it is a compile error. This rule prevents reading garbage memory.

Type conversion: widening and narrowing

```
int i = 100;
long big = i;           // widening – automatic, small → big
double d = i;           // widening – automatic

double pi = 3.99;
int cut = (int) pi;     // narrowing – needs a cast, and it TRUNCATES
System.out.println(cut);
```

Output:

3

Widening is always safe, so Java does it silently. Narrowing can lose data, so Java insists you write a **cast** (`int`) to say "I know, do it anyway." Note that casting a `double` to `int` cuts off the decimals — it does not round.

Operators you will use daily

Group	Operators	Note
Arithmetic	<code>+ - * / %</code>	<code>%</code> is remainder: <code>7 % 3 = 1</code>
Assignment	<code>= += -= *= /=</code>	<code>x += 5</code> is <code>x = x + 5</code>
Comparison	<code>== != > < >= <=</code>	yields a <code>boolean</code>
Logical	<code>&& !</code>	<code>&& / </code> short-circuit
Increment	<code>++ --</code>	pre vs post matters, see below
Ternary	<code>cond ? a : b</code>	compact if-else

The pre/post increment trap

```
int a = 5;
System.out.println(a++); // prints 5, THEN a becomes 6
System.out.println(++a); // a becomes 7 FIRST, prints 7
```

Output:

5

7

Integer division surprise

```
System.out.println(5 / 2);           // both ints → integer division
System.out.println(5 / 2.0);        // one double → decimal result
```

Output:

```
2
2.5
```

Rule of thumb

When both operands are integers, Java keeps the result an integer and throws away the fraction. To get decimals, make at least one operand a `double`.

Short-circuit logic saves you from crashes

```
String name = null;
if (name != null && name.length() > 3) { // safe
    System.out.println("Long name");
}
// name.length() is never reached when name is null.
```

Control Flow — Decisions & Loops

Programs are only useful when they can choose and repeat. That is all control flow is.

If / else if / else

```
int marks = 72;
if (marks >= 90) {
    System.out.println("Grade A");
} else if (marks >= 70) {
    System.out.println("Grade B");
} else if (marks >= 50) {
    System.out.println("Grade C");
} else {
    System.out.println("Fail");
}
```

Output:
Grade B

Java tests the conditions top to bottom and stops at the first one that is `true`. The `else` is the fallback when nothing matched.

Switch — clean multi-way choice

```
int day = 3;
switch (day) {
    case 1: System.out.println("Monday"); break;
    case 2: System.out.println("Tuesday"); break;
    case 3: System.out.println("Wednesday"); break;
    default: System.out.println("Other day");
}
```

Output:
Wednesday

Don't forget break

Without `break`, execution "falls through" into the next case and keeps printing. Modern Java offers the arrow form which never falls through: `case 3 -> System.out.println("Wednesday");`

The for loop

```
for (int i = 1; i <= 5; i++) {
    System.out.print(i + " ");
}
```

Output:
1 2 3 4 5

Three parts inside the brackets, separated by semicolons: **initialise** (`i = 1`), **condition** (`i <= 5`), and **update** (`i++`). The condition is checked before each round; the loop stops when it becomes false.

while and do-while

```
int n = 3;
while (n > 0) {           // checks first, may run zero times
    System.out.print(n + " ");
    n--;
}

int m = 0;
do {                     // runs at least once
    System.out.print("ran ");
    m++;
} while (m < 0);
```

Output:
3 2 1 ran

while vs do-while

`while` checks the condition before the body, so the body may run zero times. `do-while` checks after, so the body always runs at least once — exactly the output above.

break and continue

```
for (int i = 1; i <= 6; i++) {
    if (i == 4) break;        // stop the whole loop
    System.out.print(i + " ");
}
System.out.println();
for (int i = 1; i <= 6; i++) {
    if (i % 2 == 0) continue; // skip this round only
    System.out.print(i + " ");
}
```

Output:
1 2 3
1 3 5

Enhanced for-each loop

```
int[] nums = {10, 20, 30};
int sum = 0;
for (int value : nums) {
    sum += value;
}
System.out.println("Sum = " + sum);
```

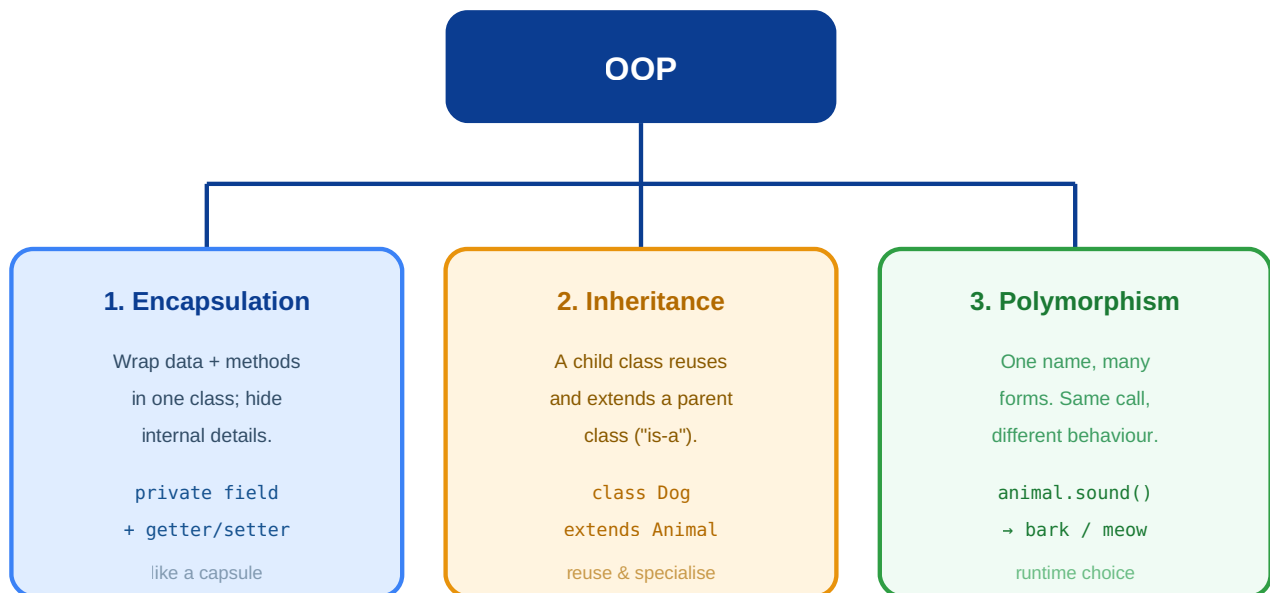
```
Output:
```

```
Sum = 60
```

The for-each loop reads each element in turn without you managing an index. It is cleaner and less error-prone, but it cannot modify the array or remove elements — for that, use a classic indexed loop or an [Iterator](#).

The Four Pillars of OOP

Object-Oriented Programming is a way of organising code around **objects** — bundles of data plus the behaviour that works on that data — instead of loose functions. Java is built entirely on four ideas: encapsulation, inheritance, polymorphism and abstraction.



4. Abstraction — show WHAT, hide HOW (abstract classes & interfaces)

Figure 4.1 — The four pillars. Abstraction sits on top: it lets you program against a simple contract.

Pillar 1 — Encapsulation

Encapsulation means keeping an object's data **private** and letting the outside world touch it only through methods you control. The word "encapsulate" literally means "wrap in a capsule".

```

class BankAccount {
    private double balance;           // hidden from outside

    public void deposit(double amount) {
        if (amount > 0) balance += amount; // validation lives here
    }

    public double getBalance() {       // controlled read
        return balance;
    }
}
  
```

Why bother? Because a caller can never put the object into an invalid state. Nobody can write `account.balance = -9999`; every change must pass through `deposit()`, which can refuse bad input. This is the single biggest source of bug-resistance in Java code.

In practice

Make fields `private`, expose behaviour through `public` methods. Expose a getter only if something genuinely needs to read the value, and a setter only if it may legally change it.

Pillar 2 — Inheritance

Inheritance lets a class inherit the fields and methods of another using the keyword `extends`. It models an "is-a" relationship: a Dog is an Animal, so a Dog automatically gets everything an Animal has.

```
class Animal {
    void eat() {
        System.out.println("This animal eats food");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();    // inherited from Animal
        d.bark();   // Dog's own method
    }
}
```

Output:

```
This animal eats food
Dog barks
```

Pillar 3 — Polymorphism

Polymorphism means "many forms". The same method call behaves differently depending on the actual object behind it. Java gives you two kinds:

- **Compile-time (overloading)** — same method name, different parameter lists. The compiler picks which one to call.
- **Runtime (overriding)** — a child replaces a parent's method. The JVM picks the version based on the real object at runtime.

```
class Animal {
    void sound() { System.out.println("Some sound"); }
}

class Dog extends Animal {
    @Override void sound() { System.out.println("Bark"); }
}

class Cat extends Animal {
    @Override void sound() { System.out.println("Meow"); }
}

public class Main {
```

```
public static void main(String[] args) {  
    Animal a;           // parent reference  
    a = new Dog(); a.sound();  
    a = new Cat(); a.sound();  
}  
}
```

Output:

```
Bark  
Meow
```

Notice the reference type stays `Animal` the whole time, yet the output changes. The object's real type decides which `sound()` runs. That is dynamic method dispatch, the heart of runtime polymorphism.

Pillar 4 — Abstraction

Abstraction hides complexity and shows only the essential contract. When you drive a car you use a steering wheel and pedals; you do not think about fuel injection. In Java you express this with **abstract classes** and **interfaces** — you declare what a thing can do, and let subclasses decide how.

```
abstract class Shape {  
    abstract double area();    // WHAT, no HOW  
}  
  
class Circle extends Shape {  
    double r = 2;  
    @Override double area() { return 3.14 * r * r; }  
}
```

Pillars work together

Abstraction says "every Shape has an area". Inheritance provides the shared code. Polymorphism lets you call `shape.area()` without knowing which shape. Encapsulation keeps each shape's internals private. One system, four ideas.

Classes, Objects & Constructors

A **class** is a blueprint. An **object** is one real thing built from that blueprint. One blueprint, many houses.

Anatomy of a class

```
class Student {  
    // 1. fields (state)  
    String name;  
    int age;  
  
    // 2. constructor (how to build one)  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    // 3. method (behaviour)  
    void introduce() {  
        System.out.println("Hi, I am " + name + ", age " + age);  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Student s1 = new Student("Ravi", 20);  
        Student s2 = new Student("Mira", 22);  
        s1.introduce();  
        s2.introduce();  
    }  
}
```

Output:

```
Hi, I am Ravi, age 20  
Hi, I am Mira, age 22
```

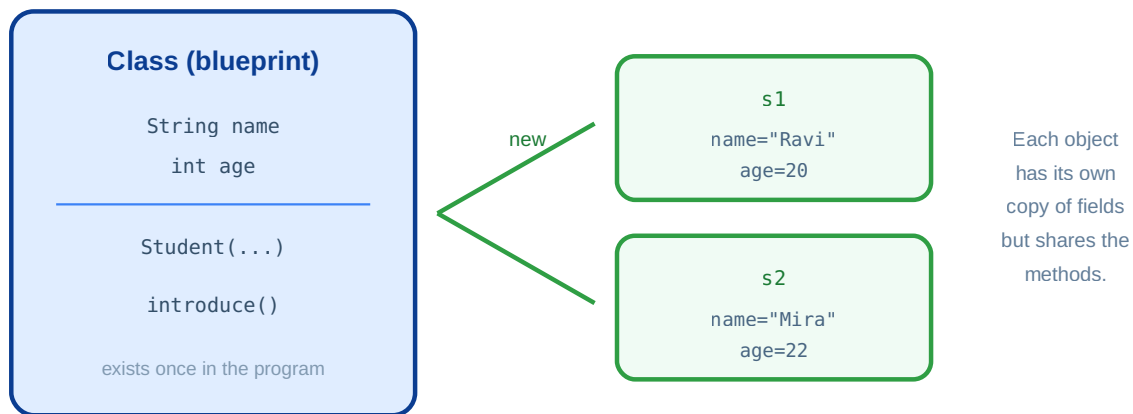


Figure 5.1 — One class, many objects. Each object owns its field values; the code is shared.

Constructors — the birth of an object

A constructor is a special method with the **same name as the class** and **no return type**. It runs exactly once, when you write `new`. Its job is to put the new object into a valid starting state.

- **Default constructor** — if you write none, Java quietly adds an empty one.
- **Parameterised constructor** — takes arguments so you can set values at creation.
- **Copy constructor** — not built in; you write one that takes another object of the same class.

```
class Book {
    String title;
    int pages;

    Book() {
        // no-arg
        this("Untitled", 0); // calls the other constructor
    }
    Book(String title, int pages) {
        this.title = title;
        this.pages = pages;
    }
    Book(Book other) {
        // copy
        this(other.title, other.pages);
    }
}
```

Constructor chaining

`this(...)` calls another constructor in the same class; `super(...)` calls the parent's constructor. Whichever you use must be the very first statement in the constructor body.

Method overloading — same name, different inputs

```
class Printer {
    void print(int x)      { System.out.println("int: " + x); }
    void print(double x)   { System.out.println("double: " + x); }
```

```

    void print(String x)    { System.out.println("String: " + x); }
}

public class Main {
    public static void main(String[] args) {
        Printer p = new Printer();
        p.print(5);
        p.print(2.5);
        p.print("Hi");
    }
}

```

```

Output:
int: 5
double: 2.5
String: Hi

```

Overloading is compile-time polymorphism: the compiler chooses by matching the argument types. Overloading is decided by parameters only — changing just the return type does not count and will not compile.

The Object class is the ancestor of everything

Every class in Java secretly extends `java.lang.Object`, which gives all objects a few useful methods you can override:

Method	Purpose
<code>toString()</code>	Text form of the object; printed by <code>println</code> .
<code>equals(Object o)</code>	Logical equality (default is identity).
<code>hashCode()</code>	Integer used by hash-based collections.
<code>getClass()</code>	Runtime type information.
<code>clone()</code>	Shallow copy.

Contract to remember

If you override `equals()`, you **must** override `hashCode()` too. Two objects that are equal must return the same hash code, or collections like `HashMap` will misbehave.

static, final, this & super

These four keywords confuse beginners because they look like modifiers but change meaning. Learn what each one really asks the compiler to do.

static — belongs to the class, not the object

A `static` member is created **once** and shared by all objects of the class. You call it on the class name, not on an instance.

```
class Counter {
    static int total = 0;    // one copy, shared
    int id;

    Counter() {
        total++;
        id = total;
    }
}

public class Main {
    public static void main(String[] args) {
        new Counter();
        new Counter();
        new Counter();
        System.out.println("Objects made: " + Counter.total);
    }
}
```

Output:
Objects made: 3

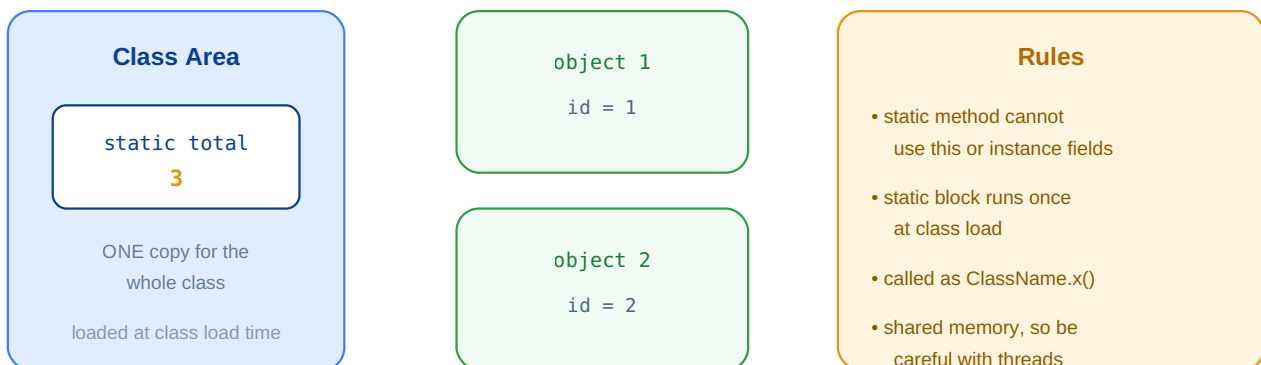


Figure 6.1 — static lives in the class, instance fields live in each object.

```
class Demo {
    static { System.out.println("static block: runs once"); }
    Demo() { System.out.println("constructor: runs per object"); }
}
```

```

}
// new Demo(); new Demo();
// Output:
// static block: runs once
// constructor: runs per object
// constructor: runs per object

```

final — cannot be changed

`final` means "assign once, then locked." It applies to three different things with three different meanings:

Used on	Meaning
Variable	A constant — value can be assigned only once.
Method	Cannot be overridden by a subclass.
Class	Cannot be extended (e.g. <code>String</code> , <code>Integer</code>).

```

final int MAX = 10;
// MAX = 20;           // compile error

class Base {
    final void rule() {} // subclasses cannot change this
}
// final class Locked {} // nobody can extend Locked

```

final reference ≠ final object

With a `final List<String> list`, you cannot point `list` at a different list — but you can still add and remove items inside it. `final` freezes the variable, not the object's contents.

this — the current object

`this` is a reference to the object the method was called on. It is used mainly to (1) disambiguate a field from a parameter of the same name, (2) chain constructors, and (3) pass the current object to another method.

```

class Point {
    int x, y;
    Point(int x, int y) {
        this.x = x; // this.x is the field, x is the parameter
        this.y = y;
    }
    Point() {
        this(0, 0); // constructor chaining
    }
}

```

super — the parent

`super` refers to the immediate parent class. Use it to call a parent method you have overridden, or to call a parent constructor.

```
class Animal {
    Animal() { System.out.println("Animal created"); }
    void speak() { System.out.println("Animal speaks"); }
}
class Dog extends Animal {
    Dog() {
        super(); // call parent constructor (implicit if omitted)
        System.out.println("Dog created");
    }
    @Override void speak() {
        super.speak(); // run the parent's version first
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        new Dog().speak();
    }
}
```

Output:

```
Animal created
Dog created
Animal speaks
Dog barks
```

Constructor order

When an object is created, the parent constructor always runs before the child's. The parent's part of the object must be built first, because the child may depend on it.

Inheritance, Abstract Classes & Interfaces

Inheritance reuses code. Abstract classes share partial implementations. Interfaces define pure contracts. Choosing between them is a skill interviewers love to test.

Types of inheritance in Java

Java supports single inheritance for classes (a class extends exactly one parent) but multiple inheritance for interfaces (an interface can extend several). The reason classes allow only one parent is the **diamond problem**: if two parents both defined the same method, the compiler would not know which to use.

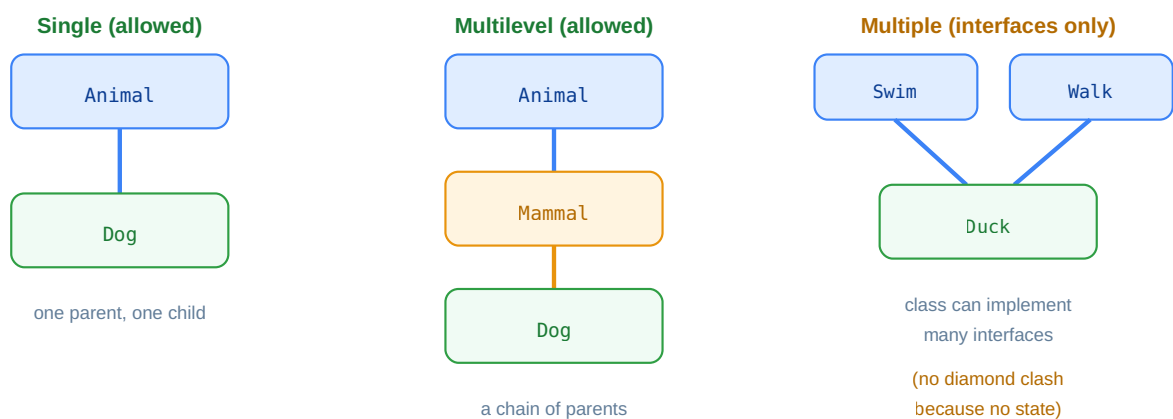


Figure 7.1 — Class inheritance is single; interface inheritance can be multiple.

Abstract classes

An abstract class is a **half-built class**. It can contain complete methods (with bodies) and abstract methods (without bodies). You cannot create an object of it directly — subclasses must fill in the abstract parts.

```
abstract class Vehicle {
    String brand;

    Vehicle(String brand) { this.brand = brand; }

    void start() {
        // concrete – shared code
        System.out.println(brand + " starting...");
    }

    abstract void fuel(); // abstract – subclass must define
}

class Car extends Vehicle {
    Car(String b) { super(b); }
    @Override void fuel() { System.out.println("Filling petrol"); }
}
```

```

}

public class Main {
    public static void main(String[] args) {
        Car c = new Car("Toyota");
        c.start();
        c.fuel();
    }
}

```

```

Output:
Toyota starting...
Filling petrol

```

Interfaces

An interface is a **pure contract**. Traditionally it held only method signatures and constants; modern Java lets it hold `default`, `static` and `private` methods too. A class "signs the contract" with `implements` and promises to provide every method.

```

interface Playable {
    void play();                                // public abstract by default
    default void pause() {                      // has a body
        System.out.println("Paused");
    }
    static void info() {
        System.out.println("Playable v1");
    }
}

class MusicPlayer implements Playable {
    @Override public void play() {
        System.out.println("Playing music");
    }
}

public class Main {
    public static void main(String[] args) {
        MusicPlayer m = new MusicPlayer();
        m.play();
        m.pause();                             // inherited default method
        Playable.info();                       // static, called on the interface
    }
}

```

```

Output:
Playing music
Paused
Playable v1

```

Abstract class vs interface — the classic question

Point	Abstract class	Interface
Keyword	<code>extends</code>	<code>implements</code>

How many	One only	Many
Fields	Any type, any modifier	Constants only (<code>public static final</code>)
Constructors	Yes	No
Method bodies	Concrete + abstract	Abstract + <code>default</code> / <code>static</code>
Access modifiers	All allowed	Methods public by default
Best when	Classes share real code/state	Unrelated classes share a capability

Interview-ready one-liner

Use an **abstract class** when you are modelling "is-a" with shared implementation. Use an **interface** when you are modelling "can-do" — a capability that many unrelated types could have (Comparable, Runnable, Serializable).

Method overriding — rules to respect

- Same method name, same parameters, same (or covariant) return type.
- The access modifier cannot become more restrictive (public → private is illegal).
- You cannot override `final`, `static` or `private` methods.
- Always write `@Override`: the compiler then catches typos like a wrong parameter list, which would otherwise silently become an overload.

Packages & Access Modifiers

A package groups related classes into a namespace; access modifiers decide who is allowed to see what.

Packages — folders for classes

```
package com.shop.model;

public class Product {
    public String name;
}
```

The first line declares the package, and the file must sit in a folder path that matches: `com/shop/model/Product.java`. Packages prevent name clashes (two teams can both have a `User` class) and control access.

```
import java.util.ArrayList;    // import one class
import java.util.*;           // import a whole package
import java.util.Scanner;
```

The four access levels

Java's access control is best remembered as a series of widening circles, from most restricted to most open.

Modifier	Same class	Same package	Subclass (other pkg)	World
<code>private</code>	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
<code>protected</code>	✓	✓	✓	✗
<code>public</code>	✓	✓	✓	✓

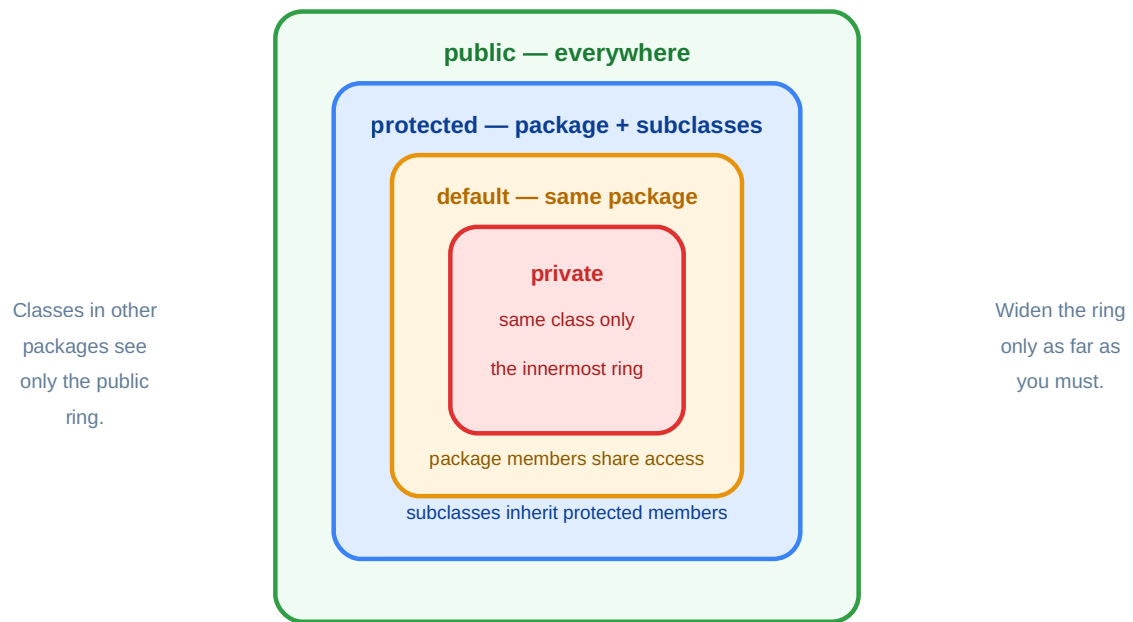


Figure 8.1 — Access modifiers as nested rings: private is the tightest, public the widest.

Design principle

Start with the tightest access that works and loosen only when a real need appears. Wide-open fields are the source of most accidental coupling.

String & Immutability

String is used more than any other class in Java, and it hides three big ideas: immutability, the string pool, and the difference between `==` and `equals()`.

Strings are immutable

Once a `String` object is created, its characters can never change. Every "modifying" method actually returns a **brand-new** object and leaves the original untouched.

```
String s = "Hello";
String t = s.concat(" World");
System.out.println("s = " + s);    // unchanged
System.out.println("t = " + t);    // new object
System.out.println(s == t);        // different objects
```

```
Output:
s = Hello
t = Hello World
false
```

Why did Java make String immutable?

- **Safety** — a String used as a file path or network address cannot be silently altered by another thread.
- **Caching / pooling** — identical literals can share one object, saving memory.
- **Hashing** — a fixed hash code makes Strings perfect keys in `HashMap`.
- **Thread safety** — immutable objects are automatically safe to share between threads.

The String constant pool

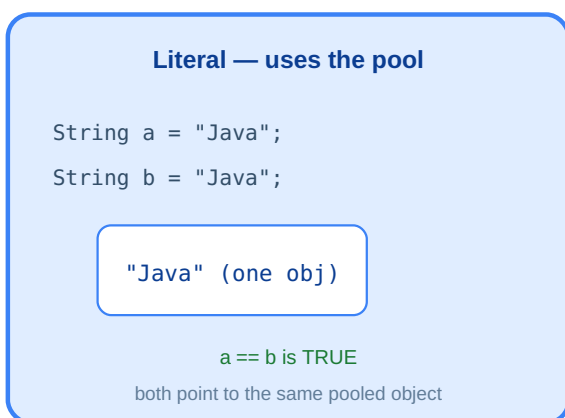


Figure 9.1 — String literals are pooled and shared; `new String(...)` always makes a fresh object on the heap.

`==` vs `equals()`

`==` compares **addresses** (are these the same object?). `equals()` compares **content** (are the characters the same?). For Strings you almost always want `equals()`.

```
String a = "Java";
String c = new String("Java");
System.out.println(a == c);           // address compare
System.out.println(a.equals(c));      // content compare
```

Output:

```
false
true
```

String vs StringBuilder vs StringBuffer

Because String is immutable, joining thousands of pieces in a loop creates thousands of objects. Use a mutable builder instead.

Class	Mutable?	Thread-safe?	Use when
String	No	Yes (immutable)	Text rarely changes
StringBuilder	Yes	No	Heavy edits, single thread (fastest)
StringBuffer	Yes	Yes (synchronised)	Heavy edits across threads

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 3; i++) sb.append(i).append(" ");
System.out.println(sb.toString().trim());
System.out.println("length = " + sb.length());
```

Output:

```
1 2 3
length = 6
```

Everyday String methods

```
System.out.println("Hello".length());           // 5
System.out.println("Hello".charAt(1));          // e
System.out.println("Hello".substring(1, 4));    // ell
System.out.println("Hello".toUpperCase());     // HELLO
System.out.println(" hi ".trim());              // hi
System.out.println("a,b,c".split(",").length); // 3
System.out.println("Hello".contains("ell"));   // true
```

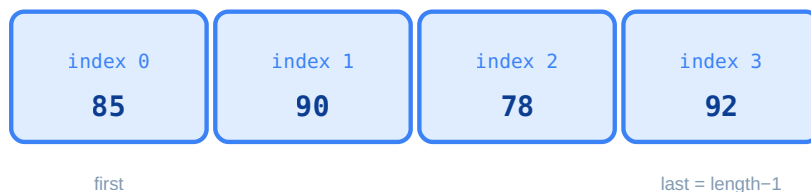
Arrays

An array is a fixed-size, numbered row of boxes holding values of one type. It is the simplest container Java offers.

Creating arrays

```
int[] a = new int[5];           // five zeros
int[] b = {10, 20, 30};        // with values
int[] c = new int[]{1, 2, 3};  // explicit new
String[] names = new String[3]; // three nulls
```

`int[] marks = {85, 90, 78, 92};` — index starts at 0



`marks.length` is 4. Valid indices are 0, 1, 2, 3 — never 4.

`marks[4]` throws `ArrayIndexOutOfBoundsException`.

Figure 10.1 — Arrays are zero-indexed; the last valid index is always `length - 1`.

Walking through an array

```
int[] nums = {5, 3, 9, 1};
int max = nums[0];
for (int i = 1; i < nums.length; i++) {
    if (nums[i] > max) max = nums[i];
}
System.out.println("Max = " + max);

for (int n : nums) System.out.print(n + " ");
```

Output:

```
Max = 9
5 3 9 1
```

2D arrays — arrays of arrays

```
int[][] grid = {
    {1, 2, 3},
    {4, 5, 6}
};
System.out.println(grid[1][2]); // row 1, column 2

for (int[] row : grid) {
```

```
    for (int v : row) System.out.print(v + " ");  
    System.out.println();  
}
```

Output:

```
6  
1 2 3  
4 5 6
```

Arrays, copying and the Arrays helper

```
import java.util.Arrays;  
  
int[] a = {3, 1, 2};  
int[] copy = Arrays.copyOf(a, a.length);  
Arrays.sort(a);  
System.out.println(Arrays.toString(a)); // [1, 2, 3]  
System.out.println(Arrays.binarySearch(a, 2)); // 1  
int[] filled = new int[3];  
Arrays.fill(filled, 7);  
System.out.println(Arrays.toString(filled));  
System.out.println(Arrays.equals(a, copy));
```

Output:

```
[1, 2, 3]  
1  
[7, 7, 7]  
false
```

Fixed size

An array's length is decided once, at creation, and can never change. When you need a growable list, use `ArrayList` (next chapters). Also remember that assigning one array variable to another copies the reference, not the contents.

Exception Handling

An exception is a problem that interrupts normal flow. Java forces you to think about them, so programs fail gracefully instead of crashing silently.

What happens without handling

```
public class Main {  
    public static void main(String[] args) {  
        int a = 10, b = 0;  
        System.out.println(a / b);  
    }  
}
```

Output:

```
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at Main.main(Main.java:4)
```

The program stops dead at the offending line. Nothing after it runs.

try / catch / finally

```
public class Main {  
    public static void main(String[] args) {  
        try {  
            int[] arr = {1, 2, 3};  
            System.out.println(arr[5]);           // problem here  
        } catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Caught: bad index");  
        } finally {  
            System.out.println("finally always runs");  
        }  
        System.out.println("program continues");  
    }  
}
```

Output:

```
Caught: bad index  
finally always runs  
program continues
```

try wraps the risky code. **catch** handles a specific exception. **finally** runs no matter what — even if an exception escapes — which makes it ideal for closing files or connections.

The exception family tree

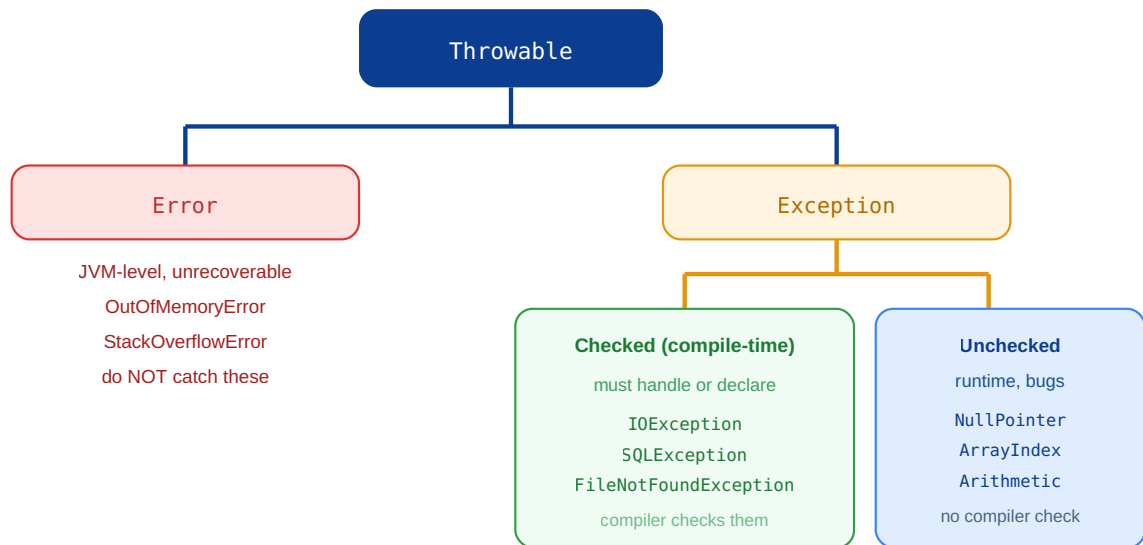


Figure 11.1 — The hierarchy of throwables: Error (don't catch), checked, and unchecked exceptions.

Checked vs unchecked — the rule

Checked exceptions extend `Exception` but not `RuntimeException`. The compiler will not let you compile until you either wrap them in try/catch or declare them with `throws`. **Unchecked exceptions** extend `RuntimeException` and represent programming bugs; the compiler does not force you to handle them.

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        throw new IOException("declared, not handled here");
    }
}
```

A realistic, safe program

```
public class Main {
    static int divide(int a, int b) {
        try {
            return a / b;
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
            return -1;
        } finally {
            System.out.println("done a=" + a + " b=" + b);
        }
    }
    public static void main(String[] args) {
        System.out.println(divide(20, 4));
        System.out.println(divide(7, 0));
    }
}
```

```
}  
}
```

Output:

```
done a=20 b=4  
5  
Cannot divide by zero  
done a=7 b=0  
-1
```

throw, throws and custom exceptions

```
class AgeException extends Exception {  
    AgeException(String msg) { super(msg); }  
}  
  
public class Main {  
    static void checkAge(int age) throws AgeException {  
        if (age < 18) throw new AgeException("Too young: " + age);  
        System.out.println("Welcome, age " + age);  
    }  
    public static void main(String[] args) {  
        try {  
            checkAge(15);  
            checkAge(21);  
        } catch (AgeException e) {  
            System.out.println("Blocked: " + e.getMessage());  
        }  
    }  
}
```

Output:

```
Blocked: Too young: 15
```

throw vs throws

`throw` is a statement that actually fires one exception object. `throws` is a claim in the method signature that this method might pass one up to its caller.

try-with-resources — automatic cleanup

```
import java.io.*;  
  
public class Main {  
    public static void main(String[] args) {  
        try (BufferedReader br =  
            new BufferedReader(new StringReader("line one"))) {  
            System.out.println(br.readLine());  
        } catch (IOException e) {  
            System.out.println("IO problem");  
        }  
    }  
}
```

```
Output:  
line one
```

Any resource declared inside the parentheses of `try(...)` is closed automatically when the block ends, even if an exception is thrown. This removes an entire class of resource leaks.

Collections Framework

Arrays are fixed; real programs need containers that grow, shrink, sort and search. The Collections Framework is Java's toolbox of them.

The big picture

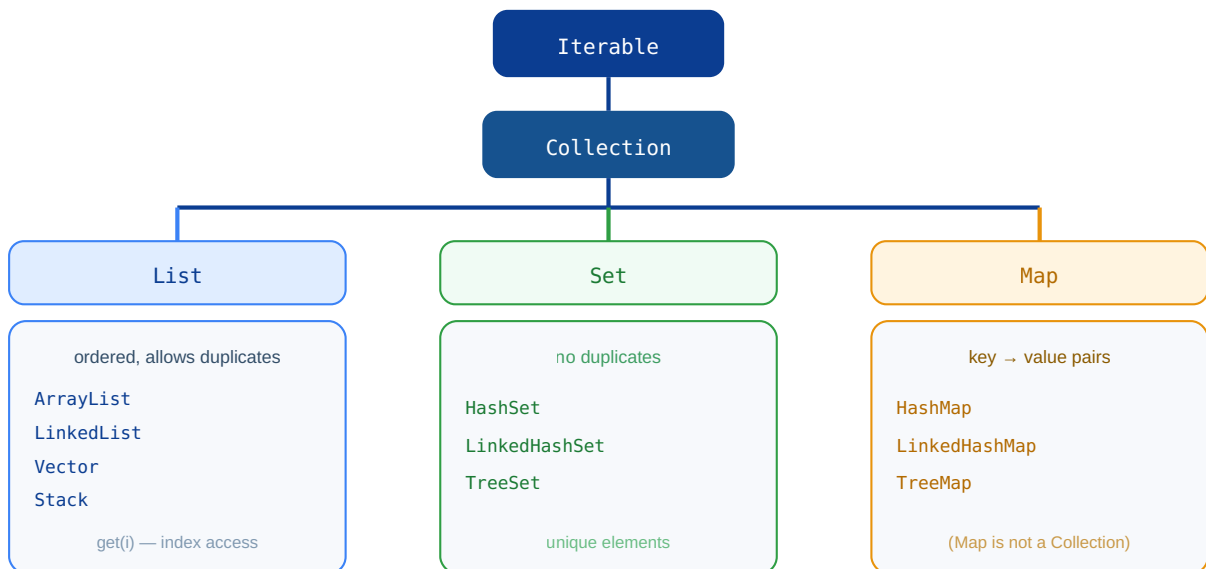


Figure 12.1 — The Collections hierarchy. Map sits beside Collection, not under it.

List — ordered, duplicates allowed

```

import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<String> fruits = new ArrayList<>();
        fruits.add("Apple");
        fruits.add("Banana");
        fruits.add("Apple");           // duplicate is fine
        fruits.add(1, "Mango");        // insert at index 1

        System.out.println(fruits);
        System.out.println("size = " + fruits.size());
        System.out.println("get(0) = " + fruits.get(0));
        System.out.println("contains Mango = " + fruits.contains("Mango"));
        fruits.remove("Apple");
        System.out.println("after remove = " + fruits);
    }
}
  
```

Output:

```

[Apple, Mango, Banana, Apple]
size = 4
  
```

```
get(0) = Apple
contains Mango = true
after remove = [Mango, Banana, Apple]
```

ArrayList is backed by an array that grows automatically; index access is instant, but inserting in the middle shifts elements. **LinkedList** uses linked nodes: cheap inserts at the ends, slow random access.

Set — no duplicates

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Set<Integer> nums = new HashSet<>();
        nums.add(5); nums.add(1); nums.add(5); nums.add(3);
        System.out.println(nums);           // no order, no dupes

        Set<Integer> sorted = new TreeSet<>(nums);
        System.out.println(sorted);         // ascending order
    }
}
```

Output:

```
[1, 3, 5]
[1, 3, 5]
```

HashSet gives fast look-ups with no order. **LinkedHashSet** keeps insertion order. **TreeSet** keeps elements sorted, at slightly higher cost.

Map — key/value pairs

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Map<String, Integer> ages = new HashMap<>();
        ages.put("Ravi", 20);
        ages.put("Mira", 22);
        ages.put("Ravi", 21);           // overwrites the old value
        System.out.println(ages);
        System.out.println("Ravi = " + ages.get("Ravi"));
        System.out.println("has Sam = " + ages.containsKey("Sam"));

        for (Map.Entry<String, Integer> e : ages.entrySet()) {
            System.out.println(e.getKey() + " -> " + e.getValue());
        }
    }
}
```

Output:

```
{Ravi=21, Mira=22}
Ravi = 21
has Sam = false
```

```
Ravi -> 21
Mira -> 22
```

A key can appear only once

Mapping the same key again replaces the value. A `HashMap` allows one `null` key and many `null` values; a `Hashtable` allows neither.

Iterator — safe removal while looping

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> n = new ArrayList<>(Arrays.asList(1,2,3,4,5,6));
        Iterator<Integer> it = n.iterator();
        while (it.hasNext()) {
            if (it.next() % 2 == 0) it.remove();    // safe
        }
        System.out.println(n);
    }
}
```

Output:
[1, 3, 5]

Don't modify during for-each

Calling `list.remove()` inside a for-each loop throws `ConcurrentModificationException`. Use an `Iterator` (or `removeIf`) instead.

Comparing and sorting

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> v = new ArrayList<>(Arrays.asList(40, 10, 30, 20));
        Collections.sort(v);
        System.out.println("asc = " + v);
        Collections.sort(v, Collections.reverseOrder());
        System.out.println("desc = " + v);
        System.out.println("max = " + Collections.max(v));
        System.out.println("min = " + Collections.min(v));
    }
}
```

Output:

```
asc = [10, 20, 30, 40]
desc = [40, 30, 20, 10]
max = 40
min = 10
```

Choosing the right collection

Need	Use	Why
Indexed list, mostly reading	<code>ArrayList</code>	Fast random access
Frequent insert/delete at ends	<code>LinkedList</code>	Cheap node linking
No duplicates, order irrelevant	<code>HashSet</code>	Fastest membership test
Unique + sorted	<code>TreeSet</code>	Automatic ordering
Key → value lookup	<code>HashMap</code>	Constant-time get/put
Key → value, sorted keys	<code>TreeMap</code>	Ordered iteration
First-in first-out queue	<code>ArrayDeque</code>	Efficient both ends

Generics

Generics let you write a class or method once and have it work safely with many types — with the compiler checking every use.

The problem before generics

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List list = new ArrayList(); // no type info
        list.add("text");
        list.add(42);
        String s = (String) list.get(1); // compiles, then crashes
    }
}
```

Without generics the list happily stores anything, and the mistake only appears at runtime as a `ClassCastException`.

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>();
        list.add("text");
        // list.add(42); // compile-time error – caught early
        String s = list.get(0); // no cast needed
        System.out.println(s.toUpperCase());
    }
}
```

Output:
TEXT

Generic classes and methods

```
class Box<T> {
    private T value;
    void set(T value) { this.value = value; }
    T get() { return value; }
}

public class Main {
    static <T> void show(T item) {
        System.out.println("Item: " + item);
    }
    public static void main(String[] args) {
        Box<Integer> bi = new Box<>();
        bi.set(100);
        System.out.println(bi.get());
    }
}
```

```

        Box<String> bs = new Box<>();
        bs.set("hello");
        System.out.println(bs.get());

        show(3.14);
        show("any type");
    }
}

```

```

Output:
100
hello
Item: 3.14
Item: any type

```

T is a **type parameter** — a placeholder filled in when you use the class. Common names: **T** (type), **E** (element), **K/V** (key/value), **N** (number).

Bounded types

```

class Calculator<T extends Number> {
    T num;
    Calculator(T num) { this.num = num; }
    double half() { return num.doubleValue() / 2; }
}

// Calculator<String> x; // illegal: String is not a Number

```

T extends Number means "any type that is a Number or its subclass". That is what lets you safely call `num.doubleValue()`.

Wildcards

```

import java.util.*;

public class Main {
    static void printAll(List<?> list) {           // any list
        for (Object o : list) System.out.print(o + " ");
        System.out.println();
    }
    static double sum(List<? extends Number> list) { // read-only, upper bound
        double t = 0;
        for (Number n : list) t += n.doubleValue();
        return t;
    }
    public static void main(String[] args) {
        printAll(Arrays.asList(1, "two", 3.0));
        System.out.println(sum(Arrays.asList(1, 2, 3, 4)));
    }
}

```

Output:

```
1 two 3.0  
10.0
```

Type erasure — what happens underneath

Generics are a **compile-time** tool. The compiler checks all your type arguments, then erases them: `List<String>` and `List<Integer>` both become plain `List` in the bytecode. That is why you cannot do `new T()`, and why `List<int>` is illegal (primitives need wrapper classes like `Integer`).

Remember

Generics give you compile-time safety, not runtime type information. Two lists of different element types are the same class at runtime.

Multithreading & Thread Life Cycle

A thread is an independent path of execution. Multiple threads let a program do several things at once — but they also introduce the hardest bugs in Java.

Two ways to create a thread

```
class MyTask extends Thread {
    @Override public void run() {
        for (int i = 1; i <= 3; i++)
            System.out.println("Thread A: " + i);
    }
}

class MyJob implements Runnable {
    @Override public void run() {
        for (int i = 1; i <= 3; i++)
            System.out.println("Thread B: " + i);
    }
}

public class Main {
    public static void main(String[] args) {
        new MyTask().start();           // start(), not run()
        new Thread(new MyJob()).start();
    }
}
```

Possible output (order varies):

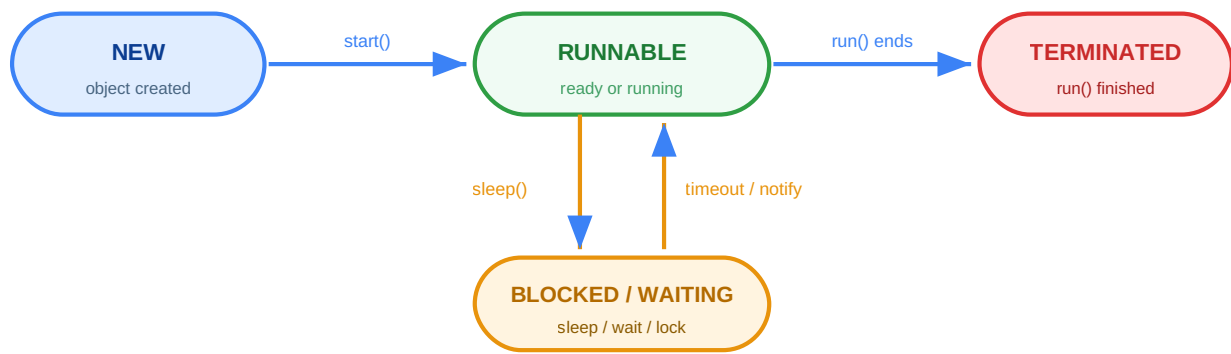
```
Thread A: 1
Thread B: 1
Thread A: 2
Thread B: 2
Thread A: 3
Thread B: 3
```

start() vs run()

`start()` creates a new thread and lets the JVM call `run()` on it. Calling `run()` directly just executes the method on the current thread — no concurrency at all. The output order above is never guaranteed; the scheduler decides.

Because Java allows only single inheritance, implementing `Runnable` is usually preferred — your class stays free to extend something else.

The thread life cycle



A thread in WAITING or BLOCKED is alive but not using the CPU.

It returns to RUNNABLE when the thing it waits for happens.

New thread state is created by `new Thread()`; `start()` hands it to the scheduler;

TERMINATED is final — you can never restart a finished thread.

Figure 14.1 — New → Runnable → (Running) → Blocked/Waiting → back to Runnable → Terminated.

Useful thread methods

```
public class Main {
    public static void main(String[] args) throws InterruptedException {
        Thread t = new Thread(() -> {
            System.out.println("Worker running");
            try { Thread.sleep(200); }
            catch (InterruptedException e) { System.out.println("interrupted"); }
        });
        t.start();
        t.join(); // main waits for t to finish
        System.out.println("Main continues after worker done");
        System.out.println("Name: " + t.getName() + " | Alive: " + t.isAlive());
    }
}
```

Output:

```
Worker running
Main continues after worker done
Name: Thread-0 | Alive: false
```

synchronized — protecting shared data

```
class Counter {
    private int count = 0;
    synchronized void increment() { count++; } // only one thread here at a time
    int get() { return count; }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {
```

```
Counter c = new Counter();
Thread t1 = new Thread() -> { for (int i=0;i<1000;i++) c.increment(); });
Thread t2 = new Thread() -> { for (int i=0;i<1000;i++) c.increment(); });
t1.start(); t2.start();
t1.join(); t2.join();
System.out.println("Final count = " + c.get());
}
}
```

Output:

Final count = 2000

Why synchronized matters

Without it, both threads can read the same old value and write back the same new one — so increments get lost and the total falls short of 2000. `synchronized` guarantees only one thread runs the method at a time, and gives "happens-before" visibility of the change to other threads.

Deadlock — when threads wait forever

Deadlock happens when thread 1 holds lock A and waits for B while thread 2 holds B and waits for A. Neither can move. The cure is discipline: always acquire multiple locks in the same order, or hold at most one lock at a time. The modern high-level tools are `ReentrantLock`, `AtomicInteger` and the `java.util.concurrent` package, which remove most hand-written `synchronized` blocks.

File I/O

Java reads and writes files through streams — either bytes (for all files) or characters (for text).

Writing a text file

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        try (BufferedWriter bw =
            new BufferedWriter(new FileWriter("notes.txt"))) {
            bw.write("First line");
            bw.newLine();
            bw.write("Second line");
        }
        System.out.println("File written");
    }
}
```

Output:
File written

Reading it back

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        try (BufferedReader br =
            new BufferedReader(new FileReader("notes.txt"))) {
            String line;
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
        }
    }
}
```

Output:
First line
Second line

The stream family

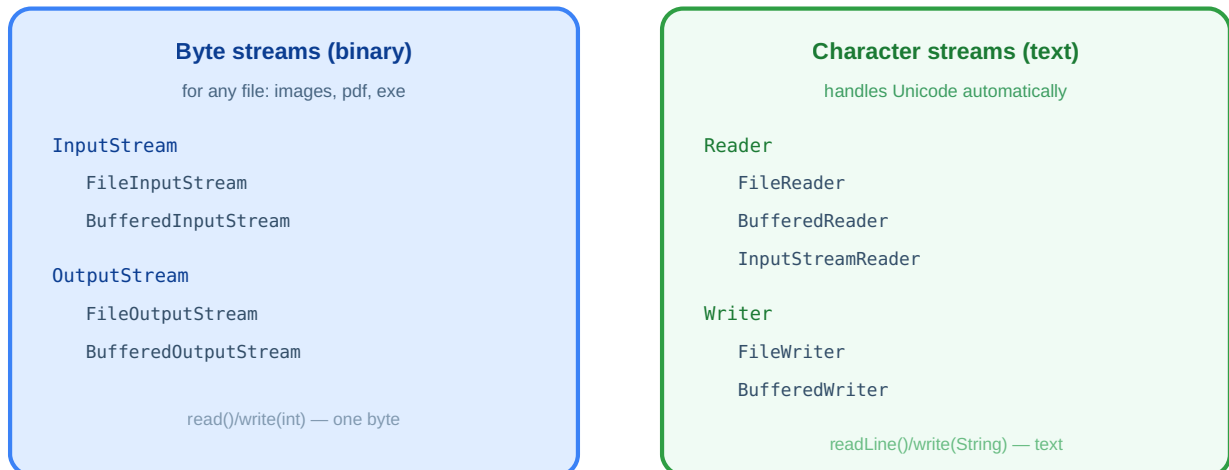


Figure 15.1 — Use byte streams for binary data and character streams for readable text.

Modern shortcut — `java.nio.file.Files`

```
import java.nio.file.*;
import java.util.List;

public class Main {
    public static void main(String[] args) throws Exception {
        Path p = Paths.get("demo.txt");
        Files.writeString(p, "Hello NIO");
        String content = Files.readString(p);
        System.out.println(content);
        System.out.println("exists = " + Files.exists(p));
        Files.write(p, List.of("a", "b", "c"));
        System.out.println(Files.readAllLines(p));
    }
}
```

Output:
Hello NIO
exists = true
[a, b, c]

Which to use?

For a couple of lines in a small file, `Files.readString` / `Files.writeString` is the shortest and safest. For large files or streaming, use buffered streams so you never load everything into memory at once.

Inner Classes

A class defined inside another class. It exists to tie a helper tightly to its owner and sometimes to reach the owner's private data.

The four kinds

Kind	Declared	Needs outer object?	Typical use
Member inner	Inside the class body	Yes	Close helper tied to an instance
Static nested	With <code>static</code>	No	Namespace grouping
Local	Inside a method	Yes	One-off helper for one method
Anonymous	At the point of use	Yes	Short one-method implementations

Member inner class

```
class Outer {
    private int value = 10;

    class Inner {
        void show() {
            System.out.println("Outer value = " + value); // reaches private field
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Outer o = new Outer();
        Outer.Inner i = o.new Inner(); // needs an outer object
        i.show();
    }
}
```

Output:
Outer value = 10

Static nested class

```
class Outer {
    static int shared = 99;

    static class Nested {
        void show() { System.out.println("Shared = " + shared); }
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        Outer.Nested n = new Outer.Nested(); // no outer object needed
        n.show();
    }
}
```

Output:
Shared = 99

Anonymous class — the quick implementation

```
interface Greeting {
    void sayHello();
}

public class Main {
    public static void main(String[] args) {
        Greeting g = new Greeting() { // no class name
            @Override public void sayHello() {
                System.out.println("Hello from anonymous class");
            }
        };
        g.sayHello();
    }
}
```

Output:
Hello from anonymous class

Where you meet them

Anonymous classes are the older way to pass behaviour — event listeners, comparators, runnables. Since Java 8 a lambda usually replaces them, but an anonymous class is still needed when you must override more than one method or hold state.

Lambdas & the Stream API

Java 8 changed how Java is written. A lambda is a compact function; streams let you describe a whole pipeline of operations on data.

A lambda is a short function

```
interface MathOp {
    int apply(int a, int b);
}

public class Main {
    public static void main(String[] args) {
        MathOp add = (a, b) -> a + b;
        MathOp mul = (a, b) -> a * b;
        System.out.println(add.apply(3, 4));
        System.out.println(mul.apply(3, 4));
    }
}
```

Output:

```
7
12
```

The syntax `(parameters) -> body` is the whole idea. A lambda only works where a **functional interface** is expected — an interface with exactly one abstract method (`MathOp` , `Runnable` , `Comparator`).

Built-in functional interfaces

Interface	Takes	Returns	Think of it as
<code>Predicate<T></code>	T	boolean	a test / filter
<code>Function<T,R></code>	T	R	a transform
<code>Consumer<T></code>	T	void	an action
<code>Supplier<T></code>	none	T	a producer
<code>Comparator<T></code>	T,T	int	an ordering

Streams — a pipeline over data

```
import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {
        List<Integer> nums = Arrays.asList(1,2,3,4,5,6,7,8,9,10);
    }
}
```

```

List<Integer> result = nums.stream()
    .filter(n -> n % 2 == 0)    // keep evens
    .map(n -> n * 10)           // multiply by 10
    .collect(Collectors.toList());

System.out.println(result);

int total = nums.stream().reduce(0, (a, b) -> a + b);
System.out.println("Sum = " + total);
System.out.println("Max = " + nums.stream().max(Integer::compare).get());
}
}

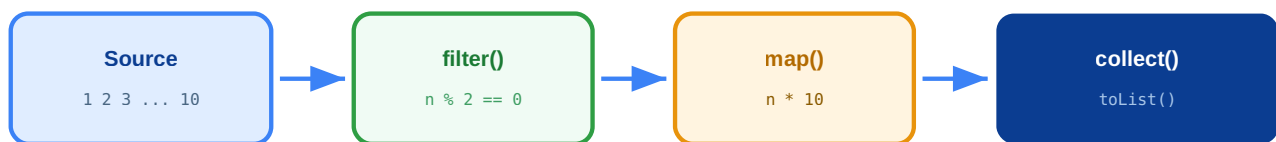
```

Output:

```

[20, 40, 60, 80, 100]
Sum = 55
Max = 10

```



Intermediate operations (filter, map, sorted) are lazy — nothing runs until a terminal operation (collect, reduce, forEach) is called.
A stream never modifies the original collection; it produces a new result.

Figure 17.1 — A stream pipeline: source → intermediate operations → terminal operation.

Common stream operations

```

import java.util.*;
import java.util.stream.*;

public class Main {
    public static void main(String[] args) {
        List<String> words = Arrays.asList("apple", "banana", "avocado", "cherry");

        words.stream().filter(w -> w.startsWith("a"))
            .forEach(System.out::println);

        long count = words.stream().filter(w -> w.length() > 5).count();
        System.out.println("long words = " + count);

        System.out.println(words.stream().sorted().collect(Collectors.toList()));
        System.out.println(words.stream().map(String::toUpperCase)
            .collect(Collectors.toList()));
    }
}

```

Output:

```

apple

```

```
avocado
long words = 2
[apple, avocado, banana, cherry]
[APPLE, BANANA, AVOCADO, CHERRY]
```

Optional — a box that may be empty

`findFirst()`, `max()` and `reduce()` can return nothing. They give an `Optional`, which forces you to state what happens when the value is absent: `opt.orElse(0)`, `opt.ifPresent(...)`. That is how Java avoids silent `NullPointerException`s in stream code.

Memory Management — Stack, Heap & Garbage Collection

Java frees you from `malloc / free`, but not from thinking about memory. Understand where things live and why leaks still happen.

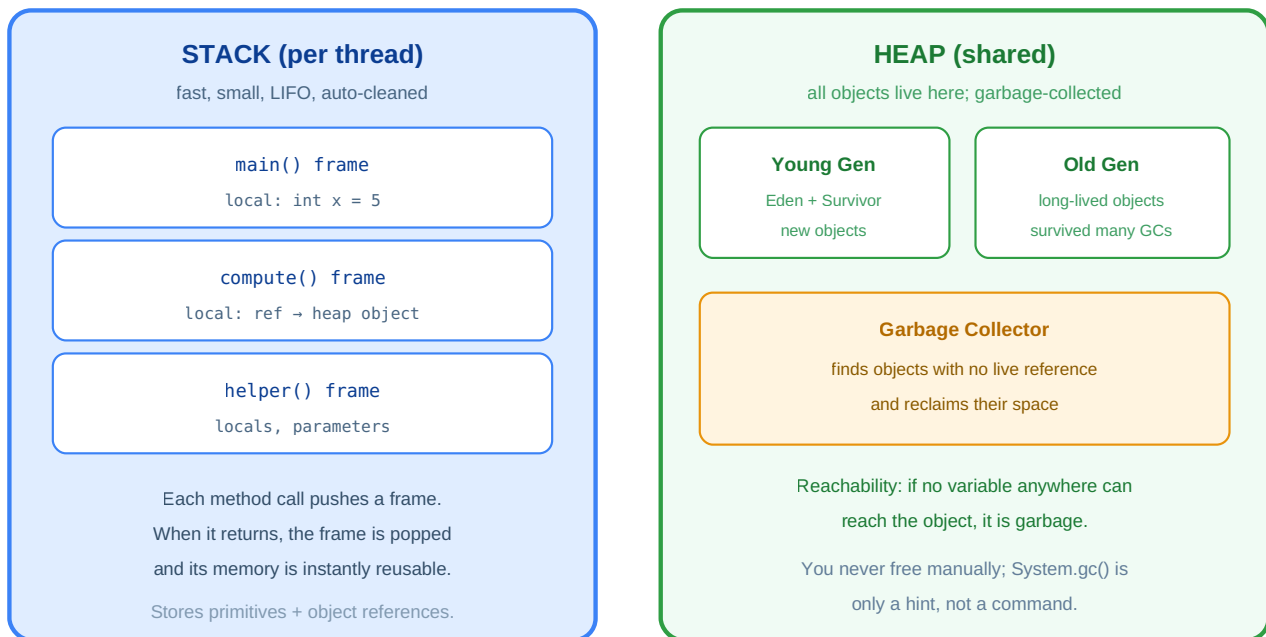


Figure 18.1 — Stack holds frames and locals; heap holds every object, cleaned up by the garbage collector.

How a variable's memory is laid out

```
class Person {
    String name;           // field – lives inside the object
}
public class Main {
    public static void main(String[] args) {
        int age = 25;      // primitive on the stack
        Person p = new Person(); // reference on stack, object on heap
        p.name = "Ravi";   // String literal in the string pool
    }
}
```

`age` holds its value right on the stack. `p` holds an address on the stack while the `Person` object — with its `name` field — rests on the heap.

How the garbage collector decides

Java does not count references (unlike old languages). It uses **reachability**: starting from the GC roots (live stack variables, static fields, JNI references), it walks every reference. Anything it cannot reach is garbage.

1. **Mark** — walk from the roots and mark everything reachable.
2. **Sweep / Compact** — delete unmarked objects and tidy the free space.
3. **Generational strategy** — most objects die young, so the young generation is collected often and cheaply; objects that survive get promoted to the old generation.

Ways to suggest cleanup

```
Person p = new Person();
p = null;           // drop the reference → object becomes eligible
System.gc();        // a HINT to the JVM, not a guarantee

class Resource implements AutoCloseable {
    @Override public void close() { System.out.println("Closed"); }
}
// try (Resource r = new Resource()) { ... } → close() called automatically
```

Memory leaks still happen in Java

- **Static collections that only grow** — a static `List` that you keep adding to but never clear lives for the whole program.
- **Listeners never removed** — the publisher keeps a reference, so the listener can never be collected.
- **Unclosed streams/connections** — the OS resource stays taken even if the object is collected.
- **Wrong equals/hashCode** — objects hide inside a `HashMap` and can never be found or removed.

Good habits

Use try-with-resources for every closeable, null out or clear long-lived collections, remove listeners when done, and prefer weak references for caches.

The essentials, in one table

Question	Answer
Where do primitives live?	On the stack if they are local; inside the object if they are fields.
Where do objects live?	Always on the heap.
Who frees the heap?	The garbage collector — automatically.
When is an object garbage?	When no live reference can reach it.
Can I force collection?	No. <code>System.gc()</code> is only a suggestion.

Where to go next

You now have the full core: the JVM, the language, objects, the collections, concurrency, I/O, streams and memory. Natural next steps are **JDBC** for databases, **Java modules and build tools** (Maven/Gradle), **unit testing** with JUnit, and finally the frameworks built on top of this foundation — Spring and Spring Boot. Every one of them assumes exactly the material in these eighteen chapters.